TALLER DE PRUEBAS UNITARIAS

Presentado por:
Samuel Moncada Salazar

Profesor:
Cesar Augusto Palacios Alarcon

Viernes 13 de Febrero

Universidad Nacional de Colombia
Facultad de Administración
Administración de Sistemas Informáticos
2026

Calculadora

1. Primer bug encontrado y solucionado → en la función mínima se encontrada esta línea de código → min_val = 0. Esto es un error debido a que si los valores de la lista son todos mayores a 0 no devolverá el menos valor encontrado si no que devolverá 0.
   Solución → min_val = float('inf') → inicia la variable como infinito positivo haciendo que la función realmente devuelva el valor esperado.

```python
def minimo(lista):
    """Encuentra el número mínimo de una lista."""
    if not lista:
        raise ValueError("La lista no puede estar vacía")
    # BUG 2: No maneja correctamente la comparación inicial
    min_val = float('inf')
    for num in lista:
        if num < min_val:
            min_val = num
    return min_val
```

2. Segundo bug → la función no devuelve la raíz cuadrada de números negativos. por la cual decidí importar la herramienta cmath la cual calcula la raíz cuadrado de números negativos, haciendo que devuelva el valor esperado.

```python
def raiz_cuadrada(numero):
    """Calcula la raíz cuadrada de un número."""
    # BUG 1: No maneja números negativos
    return cmath.sqrt(numero)
```

Fibonacci

3. Tercer bug encontrado → la función era muy lenta en caso de número grandes. Se importa la herramienta lru_cache (es una herramienta de memoizacion de python)

```python
from functools import lru_cache
```

```python
@lru_cache(maxsize=None)
def fibonacci_recursivo(n):
    """Calcula el n-ésimo número de Fibonacci de forma recursiva."""
    if n < 0:
        return "El índice no puede ser negativo"

    if n == 0:
        return 0

    if n == 1:
        return 1

    # BUG: No tiene memoización, es muy lento para n grande
    return fibonacci_recursivo(n - 1) + fibonacci_recursivo(n - 2)
```

4. cuarto bug encontrado → El impuesto se calcula incorrectamente. corrección →

```python
def calcular_precio_final(precio, impuesto, descuento):
    """Calcula el precio final con impuesto y descuento."""
    if precio < 0:
        raise ValueError("El precio no puede ser negativo")

    if impuesto < 0:
        raise ValueError("El impuesto no puede ser negativo")

    if descuento < 0 or descuento > 100:
        raise ValueError("El descuento debe estar entre 0 y 100")

    # Primero aplicar el descuento
    precio_con_descuento = aplicar_descuento(precio, descuento)

    # Luego aplicar el impuesto
    # BUG 1: El impuesto se aplica incorrectamente
    impuesto_monto = precio_con_descuento * (impuesto / 100)
    precio_final = precio_con_descuento + impuesto_monto

    return precio_final
```

5. quinto bug encontrado → es_palindromo no maneja espacios, ni mayúsculas/minúsculas. corrección:

```python
def es_palindromo(cadena):
    """Verifica si una cadena es palíndromo."""
    if not cadena:
        return False

    # BUG 1: No maneja espacios ni mayúsculas/minúsculas
    cadena_limpia = cadena.lower().replace(" ", "")
    return cadena_limpia == cadena_limpia[::-1]
```

6. sexto bug encontrado → dividir_en_palabras no maneja múltiples espacios ni puntuación. corrección:

```python
def dividir_en_palabras(cadena):
    """Divide una cadena en palabras."""
    if not cadena:
        return []

    # BUG 2: No maneja múltiples espacios ni puntuación
    return cadena.split()
```

Autenticación

7. Séptimo bug encontrado → hash_contraseña no maneja contraseñas vacias o None. corrección:

```python
def hash_contraseña(contraseña):
    """Genera un hash SHA-256 de una contraseña."""
    if not contraseña or contraseña.strip() == "":
        return "La contraseña no puede estar vacía o ser None"
    # BUG 1: No maneja contraseñas vacías o None
    return hashlib.sha256(contraseña.encode()).hexdigest()
```

8. Octavo bug encontrado → generar_token No incluye salt o aleatoriedad, el token es predecible. corrección

```python
def generar_token(usuario, timestamp):
    """Genera un token seguro con aleatoriedad."""
    nonce = secrets.token_hex(16)
    datos = f"{usuario}:{timestamp}:{nonce}"

    return hashlib.sha256(datos.encode()).hexdigest()
```

Utils

9. Noveno bug encontrado → mcm tiene un error en el calculo. correción:

```python
def mcm(a, b):
    """Calcula el Mínimo Común Múltiplo."""
    # BUG: Error en el cálculo
    if a == 0 or b == 0:
        return 0
    return a * b // mcd(a, b)
```

10. Décimo bug encontrado → parse_csv_line No maneja correctamente comillas anidadas o escapadas. corrección:

```python
def parse_csv_line(linea):
    """Parsea una línea de CSV separada por comas."""
    if not linea:
        return []

    return next(csv.reader(StringIO(linea)))
```

11. Décimo primer bug encontrado → csv_a_diccionarios No maneja filas con diferente número de columnas. correción:

```python
def csv_a_diccionarios(contenido_csv, tiene_cabecera=True):
    """Convierte contenido CSV a una lista de diccionarios."""
    lineas = leer_csv_como_lista(contenido_csv)

    if not lineas:
        return []

    if tiene_cabecera and len(lineas) > 1:
        cabeceras = lineas[0]
        datos = lineas[1:]

        if len(cabeceras) == 1 and ',' not in cabeceras[0] and ' ' in cabeceras[0]:
            cabeceras = cabeceras[0].split()

        resultado = []
        for fila in datos:
            if len(fila) == 1 and ',' not in fila[0] and ' ' in fila[0]:
                fila = fila[0].split()

            fila_dict = {}
            for i, cabecera in enumerate(cabeceras):
                fila_dict[cabecera] = fila[i] if i < len(fila) else ""
            resultado.append(fila_dict)

        return resultado
```

Resultado general de los tests: